

car-coating — User Manual

Product: car-coating (Keeper PRO SHOP multi-tenant storefront + admin builder) **Version:** 0.1.0
Date: 2026-05-07 **Tagline:** A multi-tenant Next.js storefront and drag-and-drop admin builder for Keeper PRO SHOP car-coating stores — customers browse per-store pages, request estimates, and book appointments; super admins and store admins manage their content end-to-end.

Table of Contents

1. [Introduction](#)
 2. [Getting Started](#)
 3. [User Guide \(End Users / Store Visitors\)](#)
 4. [Operator Guide \(Store Admins\)](#)
 5. [Administrator Guide \(Super Admins\)](#)
 6. [Deployment Guide](#)
 7. [Maintenance Guide](#)
 8. [Troubleshooting](#)
 9. [API Reference](#)
 10. [Glossary](#)
 11. [Changelog](#)
 12. [Support and Contact](#)
-

1. Introduction

1.1 What this product does

car-coating is the website and back-office platform for the Keeper PRO SHOP car-coating chain. A single Next.js application powers:

- A **public marketing site** (homepage at `/`, blog at `/blog`).
- One **storefront per store**, mounted at `/<slug>`, with sub-pages for pricing, options, coatings, access, reviews, news, FAQ, cases, booking, inquiries, and a privacy notice.
- An **admin dashboard** at `/admin` for super admins (platform operators) and store admins (per-store operators) to manage stores, content, bookings, inquiries, banners, blog posts, KPI, and support tickets.

- A **drag-and-drop page builder** at `/admin/builder` that composes each storefront from reusable blocks (hero, pricing, gallery, FAQ, banners, custom HTML, etc.).

1.2 Audiences

Audience	What they do	Where they go
End user (customer)	Browse a store, request an estimate, book a slot, contact the store	<code>/<slug>/...</code>
Store admin (operator)	Manage their own store(s): bookings, inquiries, page content, banners, news	<code>/admin/**</code> (filtered to <code>managed_stores</code>)
Super admin (maintainer)	Manage the platform: stores, users, global defaults, master data, blog, hierarchy, diagnostics	<code>/admin/**</code> (full access)

1.3 Key concepts and terminology

- **Store** — a single KeePer PRO SHOP branch. Identified by a URL `slug` , stored as a Firestore document under `stores/{store_id}` .
- **Sub-company** (`sub_companies/{id}`) — a parent organisation that owns multiple stores. Used for hierarchy and branding inheritance.
- **Sub-store** — a child store mounted at `/<slug>/<subSlug>` .
- **Page layout / blocks** — each store's homepage is described by a `page_layout` JSON document of typed `blocks` (`hero` , `pricing` , `gallery` , `cta` , `custom_html` , ...) rendered in order.
- **Banner preset** — a reusable banner snippet (structured, raw HTML, or parameterised template) that admins can drop onto a storefront.
- **Coating tier** — a master-data record defining a coating product (Crystal KeePer, Diamond KeePer, etc.) with sized prices and options.
- **Override / global default** — site-wide defaults can be set in `globalDefaults` ; an individual store may override any field.
- **Reservation** — a customer booking, status `pending` → `confirmed` → `completed` (or `cancelled`).
- **Inquiry** — a customer contact-form submission, status `open` → `replied` → `closed` .
- **Ticket** — an internal support ticket between store admins and the super-admin team.

1.4 Architecture overview



Stack at a glance:

Layer	Tech
Framework	Next.js 16 (App Router), React 19, TypeScript
Styling	Tailwind CSS 4
Database	Firestore via Firebase Admin SDK
Auth	Firebase Auth + 14-day <code>__session</code> cookie
Storage	Firebase Cloud Storage
Email	Nodemailer → Gmail SMTP
External APIs	Google Maps (client), Places (reviews), Calendar (booking)
Validation	Zod (<code>src/lib/validations.ts</code>)
Tests	Vitest (unit) + Playwright (e2e)
Deployment	Vercel (Fluid Compute, Node.js runtime)

2. Getting Started

2.1 System requirements

- **Node.js** ≥ 20 (Next.js 16 requirement).
- **npm** ≥ 10 (bundled with Node 20). The repo uses `package-lock.json` — do not switch to yarn/pnpm without coordination.
- **Operating system:** macOS, Linux, or WSL2 on Windows.
- **Browsers (customer-facing):** modern evergreen Chrome / Edge / Firefox / Safari (last 2 versions). No IE11 support.
- **External accounts you need before running locally:**
 - A Firebase project (Auth + Firestore + Storage enabled).
 - A Gmail account with an app password (for outbound transactional email).
 - A Google Cloud project with **Maps JavaScript API**, **Places API**, and **Calendar API** enabled.
 - (Optional) A Vercel account if you intend to deploy.

2.2 Installation

```
git clone <repo-url> car-coating
cd car-coating
npm install
cp .env.example .env.local
# Fill in .env.local — see §2.3
npm run dev
# → http://localhost:8080
```

2.3 Environment variables (first-time configuration)

All variables live in `.env.local` for development; production values are managed in Vercel via `vercel env`. The canonical list lives in `.env.example` at the repo root.

Variable	Purpose
<code>FIREBASE_PROJECT_ID</code>	Firebase Admin SDK — project ID.
<code>FIREBASE_CLIENT_EMAIL</code>	Service-account email.
<code>FIREBASE_PRIVATE_KEY</code>	Service-account private key. Paste with <code>\n</code> escapes intact.
<code>NEXT_PUBLIC_FIREBASE_API_KEY</code>	Firebase client SDK — bundled into the browser.
<code>NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN</code>	e.g. <code>your-project.firebaseio.com</code> .
<code>NEXT_PUBLIC_FIREBASE_PROJECT_ID</code>	Same as <code>FIREBASE_PROJECT_ID</code> .
<code>NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET</code>	Storage bucket for image uploads.
<code>NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID</code>	Firebase messaging sender.
<code>NEXT_PUBLIC_FIREBASE_APP_ID</code>	Firebase app ID.
<code>NEXT_PUBLIC_GOOGLE_MAPS_API_KEY</code>	Client-side Google Maps JS API key.
<code>GOOGLE_PLACES_API_KEY</code>	Server-side Places API key (Google reviews).
<code>GOOGLE_OAUTH_CLIENT_ID</code>	OAuth client — used by <code>scripts/oauth-create-calendars.ts</code> .
<code>GOOGLE_OAUTH_CLIENT_SECRET</code>	OAuth client secret.
<code>EMAIL_USER</code>	Outbound Gmail address (e.g. <code>bookings@yourdomain.jp</code>).
<code>EMAIL_APP_PASSWORD</code>	Gmail app password (NOT the account password).
<code>NEXT_PUBLIC_SITE_URL</code>	Canonical URL — used in emails, sitemap, robots, OG. Default <code>http://localhost:8080</code> .
<code>NEXT_PUBLIC_LINE_CHANNEL_ID</code>	Optional. LINE button on store pages.

If `EMAIL_USER` is unset, all email-sending helpers in `src/lib/email.ts` silently no-op — convenient for local dev and Playwright runs.

2.4 Running locally

```
npm run dev      # http://localhost:8080
npm run build    # production build (also typechecks)
npm run start    # serve the production build
npm run lint     # ESLint
npx vitest run   # unit tests
npx playwright test # e2e tests (boots dev server on :8081)
```

There are no `test:*` shortcut scripts in `package.json` — invoke `vitest` and `playwright` directly via `npx`.

2.5 Verifying the install

1. Open <http://localhost:8080>. The KeePer PRO SHOP marketing homepage should render.
2. Navigate to `/<some-slug>` for any seeded store (use `npx tsx --env-file=.env.local scripts/list-all-stores.ts` to find one).
3. Open `/login` and authenticate with a Firebase user that has the `super_admin` custom claim. You should be redirected to `/admin`.
4. From `/admin`, click **店舗マスター (Store Master)** — the list should load without errors.

If step 3 fails with “アカウントが登録されていません” the user has signed in via Firebase Auth but has no Firestore `users/{uid}` document yet. Bootstrap the first super admin via `scripts/seed-*` or by directly writing a Firestore record (see §5.1).

3. User Guide (End Users / Store Visitors)

This section describes the public-facing routes a customer uses.

3.1 Marketing homepage /

The corporate landing page. Composed of homepage-only blocks (`hero_home`, `service_menu`, `why_keeper`, `store_finder`, `blog_section`, `news_home`, `process_home`, `cta_home`) configured under `/admin/homepage`.

 Screenshot: marketing homepage

3.2 Store storefront /<slug>

Each store has a top page rendered by `src/app/[slug]/page.tsx`. The sections shown are controlled by the store's `page_layout` (set in the page builder). Out of the box, the default block

order is:

hero → store_intro → staff_photo → before_after → gallery → usp → quiz → simulator → cases → pricing → staff → concerns (FAQ) → news → process → benefits → access → cta .

Tip: the homepage URL of every active store is listed in `/sitemap.xml` .

3.3 Sub-pages on the storefront

URL	Page	Source
<code>/<slug>/access</code>	Address, map, parking, nearby stations	<code>src/app/[slug]/access/</code>
<code>/<slug>/booking</code>	Calendar + form to book a slot	<code>src/app/[slug]/booking/</code>
<code>/<slug>/cases</code>	Gallery of past coating jobs	<code>src/app/[slug]/cases/</code>
<code>/<slug>/coatings</code>	Coating-tier comparison page	<code>src/app/[slug]/coatings/</code>
<code>/<slug>/guide</code>	"How it works" walkthrough	<code>src/app/[slug]/guide/</code>
<code>/<slug>/inquiry</code>	Contact form	<code>src/app/[slug]/inquiry/</code>
<code>/<slug>/news</code>	Store news & announcements	<code>src/app/[slug]/news/</code>
<code>/<slug>/options</code>	Add-on services & options	<code>src/app/[slug]/options/</code>
<code>/<slug>/price</code>	Detailed pricing table	<code>src/app/[slug]/price/</code>
<code>/<slug>/privacy</code>	Privacy notice	<code>src/app/[slug]/privacy/</code>
<code>/<slug>/reviews</code>	Google Places-sourced reviews	<code>src/app/[slug]/reviews/</code>
<code>/<slug>/<subSlug>[/...]</code>	Sub-store hierarchy	<code>src/app/[slug]/[subSlug]/</code>

3.4 Booking a slot — step by step

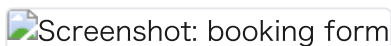
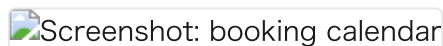
Path: store homepage → **Book / 予約** CTA → `/<slug>/booking` .

1. **Pick a date.** The calendar (`BookingCalendar.tsx`) calls `GET /api/slots?store=<id>&month=YYYY-MM` and grays out unavailable days.
2. **Pick a time.** Selecting a date calls `GET /api/slots?store=<id>&date=YYYY-MM-DD` and lists open times.
3. **Fill the form** (`ReservationForm.tsx`). Required: name, phone, email. Optional: vehicle info, notes, selected coating(s) and option(s).
4. **Submit.** The form POSTs to `/api/reservation` . By default `autoConfirm` is `true` for type: `'visit'` , so the booking is immediately `confirmed` .
5. **Confirmation email** lands in the customer's inbox (subject `【確定】ご予約確認 - <store>`), with a `.ics` calendar attachment and a cancel link of the form `/cancel/<id>?token=<cancelToken>` .

6. **Cancel** any time before the visit using the link — the customer does not need an account; the `cancelToken` authenticates the request.

Common mistakes:

- **Booking in the past.** The server rejects with HTTP 400 “Booking must be in the future”.
- **Bad email format.** Validated server-side with `^[^\s@]+@[^\s@]+\.[^\s@]+$`.
- **Hitting the rate limit.** 5 reservations per minute per IP.



3.5 Sending an inquiry / requesting an estimate

Path: store homepage → **Inquiry / お問い合わせ** CTA → `/<slug>/inquiry`, or use the dedicated estimate flow at `/estimate`.

1. Fill name, phone (optional), email, message; optionally pick a coating tier.
2. Submit → `POST /api/inquiry`.
3. The customer receives a confirmation email; staff at the store receive a notification email (per-store email list in `storeSettings/{store_id}.notificationEmails`).
4. The inquiry appears for staff under `/admin/inquiries`, status `open`.

Rate limit: 10 per minute per IP. Personal token follow-ups (when staff emails the customer back) use `/api/inquiries/[token]` to look up the record without an admin session.

3.6 Browsing reviews

`/<slug>/reviews` calls `GET /api/reviews/[placeId]` server-side, which proxies the Places API using `GOOGLE_PLACES_API_KEY` (never exposed to the browser).

3.7 Blog

`/blog` lists all published posts; `/blog/[articleSlug]` renders a single article. Posts are super-admin authored under `/admin/blog`.

4. Operator Guide (Store Admins)

Sign in at `/login`. Email + password is supported; Google sign-in is also wired up but the user record must already exist in the `users` collection (see §5.1).

After login, you land on `/admin`. The sidebar (`AdminSidebar.tsx`) shows you only the items your role grants. As a `store_admin`, you see:

- **ダッシュボード** — `/admin`. Stats and quick actions.
- **予約管理** — `/admin/bookings`. Confirm / cancel / complete bookings.

- お問い合わせ — `/admin/inquiries` . Reply to or close inquiries.
- ページビルダー — `/admin/builder` . Edit your storefront blocks.
- バナーメーカー — `/admin/banners` . Manage banner presets.
- お知らせ管理 — `/admin/news` . Per-store news entries.
- KPIダッシュボード — `/admin/kpi` . Daily phone / inquiry / booking counters.
- チケット — `/admin/tickets` . Open a ticket to the platform team.
- CSVインポート / インポート履歴 — `/admin/stores/import` , `/admin/imports` .

Store admins can only act on stores listed in their token's `managed_stores` . Server endpoints enforce this via `canManageStore(user, storeId)` from `src/lib/auth.ts` .

4.1 Confirming a booking — `/admin/bookings`

1. Find the row, status `pending` .
2. Click **承認 (confirm)**. The handler is `PATCH /api/admin/bookings` , validated with `bookingPatchSchema` .
3. On confirm:
 - Status becomes `confirmed` .
 - If `storeSettings/{storeId}.calendarId` is set, a Google Calendar event is created via `createCalendarEvent()` in `src/lib/google-calendar.ts` .
 - The customer gets the `【確定】` email + `.ics` attachment.
 - Optional `adminMessage` is included in the email body.

Other transitions are `confirmed → completed` (after the visit) and `* → cancelled` (delete-of-record, also pulls the calendar event).

4.2 Replying to an inquiry — `/admin/inquiries`

1. Open an inquiry; status `open` .
2. Type a reply and click **返信 (reply)**. The handler is `PATCH /api/admin/inquiries` , validated with `inquiryPatchSchema` .
3. The customer receives the reply by email (`sendInquiryReplyEmail()`), the inquiry status moves to `replied` . Use **クローズ (close)** to set status `closed` when done.

4.3 Editing a storefront — `/admin/builder`

The builder is a drag-and-drop editor (`@dnd-kit/sortable`) of `PageBlock[]` . Blocks are typed (see `src/lib/block-types.ts`).

1. Pick the store from `/admin/builder` (super admins) or jump straight to `/admin/builder/<storeId>` (store admins; only your stores).
2. **Reorder** blocks by dragging them in the left rail.
3. **Toggle visibility** with the eye icon (`block.visible`).

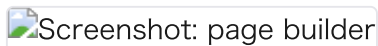
4. **Edit a block** by clicking it — a per-type editor opens (see

`src/app/admin/builder/components/editors/`):

- `hero` — title, subtitle, image, CTA toggles.
- `pricing` — featured tiers, blur fields (price-hidden until login), option-discount sync.
- `gallery` — column counts, max images.
- `staff` — heading + member list (name / role / photo / bio).
- `concerns` (FAQ) — Q&A list.
- `custom_html` — raw HTML + CSS, sanitized server-side via `sanitizeHtml()` / `sanitizeCss()`.
- `banners` — list of structured / HTML / template banners.

5. **Save.** The layout is serialized to JSON and stored on the store document's `page_layout` field.

The builder applies a small migration on load (`migrateLayout()` in `src/lib/block-types.ts`) — it auto-inserts the `staff` block on legacy layouts and snaps `concerns` after `staff` if it was still in its legacy auto-default position.



4.4 Managing banners — `/admin/banners`

Banners come in three modes (`bannerPresetWriteSchema`):

- **structured** — title, subtitle, image, original price, discount %, link URL, optional `custom_css`.
- **html** — raw `html` + `css` snippet.
- **combined** — a single `source` blob (HTML+CSS+JSON params) used by the banner-maker live-preview gallery.

Presets have a `scope`:

- `global` — visible to every store; only super admins may write them.
- `store` — owned by `owner_store_id`; that store's admins (and super admins) may write.

A preset can also be marked `is_template: true`, in which case it declares typed `fields[]` (text / textarea / color / number / select / image_url / url) that the page-builder per-banner editor uses to substitute values into the cached HTML/CSS at placement time.

4.5 Per-store store settings — `/admin/bookings/settings/store`

Sets:

- `calendarId` — the Google Calendar to push confirmed bookings into.
- `notificationEmails` — staff addresses CC'd on every booking and inquiry email.

Both are stored under `storeSettings/{storeId}` and cached in process for 5 minutes (`src/lib/store-settings.ts`).

4.6 News, KPI, tickets

- **News** — `/admin/news` . Each entry: `title` , `content` , `date` , `visible` . Rendered by the `news` block.
- **KPI** — `/admin/kpi` . Server tracks daily counters under `kpi/{storeId}/daily/{YYYY-MM-DD}` for inquiries, bookings, and tracked phone clicks (`/api/track`).
- **Tickets** — `/admin/tickets` . Open a ticket to the super-admin team. Validated via `ticketActionSchema` (`create` | `reply` | `status` | `edit` | `delete` | `delete_message`); ticket emails go through `sendTicketNotificationEmail()` .

5. Administrator Guide (Super Admins)

Super admins see every sidebar item plus extras gated by `superAdminOnly: true` :

- 店舗マスター — `/admin/stores` .
- 店舗構成図 — `/admin/hierarchy` (sub-companies / parent-child).
- 施工事例 — `/admin/cases` .
- トップページ — `/admin/homepage` (the marketing landing layout).
- グローバルデフォルト — `/admin/defaults` .
- キャンペーン — `/admin/campaigns` .
- ブログ管理 / ブログ CSV — `/admin/blog` , `/admin/blog/import` .
- マスターデータ — `/admin/master` (coating tiers, appeal points).
- 診断 — `/admin/diagnostics` .

5.1 User management — `/admin/users`

Backed by `POST /api/auth/users` (create) and `DELETE /api/auth/users/[uid]` (delete) — both gated by `requireAuth( 'super_admin' )` .

To create a user:

1. `/admin/users` → 新規作成 (New).
2. Fill email, temporary password, display name, role, and (for `store_admin`) a list of `managed_stores` the user may operate.
3. The handler calls `createUser()` from `src/lib/auth.ts` , which:
 - Creates a Firebase Auth user.
 - Sets custom claims `{ role, managed_stores }` on the user.
 - Writes the same fields to Firestore under `users/{uid}` .
4. The user can now sign in at `/login` and is automatically redirected to `/admin` .

To **bootstrap the very first super admin** when no super admin yet exists:

```
# Create a Firebase Auth user via the Firebase console or admin SDK,
# then directly write the Firestore doc and set custom claims:
npx tsx --env-file=.env.local scripts/seed-all-stores.ts
# (or use a one-off script that calls setUserClaims +
db.collection('users').doc(uid).set(...))
```

Once bootstrapped, every subsequent super admin can be created via `POST /api/admin/setup` (also gated by `requireAuth('super_admin')`).

5.2 Roles and permissions

Action	super_admin	store_admin
Read any store	yes	only managed_stores
Create / delete store	yes	no
Update store fields & layout	yes	only managed_stores
Create / edit users	yes	no
Edit master data, global defaults, campaigns, homepage, hierarchy	yes	no
Edit blog	yes	no
Approve / cancel bookings on managed stores	yes	yes (own only)
Reply to inquiries on managed stores	yes	yes (own only)
Read KPI on managed stores	yes	yes (own only)
Open / reply to tickets	yes	yes

Enforcement: `src/proxy.ts` rejects unauthenticated traffic to `/admin/**`, `/api/admin/**`, and `/api/v3/**` writes. Each handler re-checks role via `requireAuth(...)`. Firestore rules in `firestore.rules` apply the same split for direct client reads.

5.3 System settings (super-admin)

Page	What it controls
<code>/admin/defaults</code>	<code>globalDefaults</code> doc — site-wide theme, hours, defaults. Per-store overrides handled via <code>override_flags</code> JSON.
<code>/admin/master</code>	<code>master_data/coating_tiers</code> and <code>master_data/appeal_points</code> . Tiers are the canonical pricing source; per-store <code>price_overrides</code> JSON multiplies / overrides per size.
<code>/admin/homepage</code>	Layout (<code>PageLayout</code>) for the root marketing page.
<code>/admin/campaigns</code>	Campaign defaults (title, color, dates, discount %, font). Validated by <code>campaignDefaultsSchema</code> .
<code>/admin/hierarchy</code>	<code>sub_companies</code> collection.
<code>/admin/diagnostics</code>	Read-only system probes (Firebase reachable, env vars set, etc.).

5.4 Backup & restore

This product does **not** ship a built-in backup CLI. Recommended practice:

- Enable **Firestore daily backups** in Firebase / Google Cloud Console (managed export → GCS bucket).
- For lightweight, manual snapshots, use the export endpoint `GET /api/v3/stores/export` (returns CSV of every store) — restore via `POST /api/v3/stores` with `Content-Type: text/csv`.
- For per-import safety nets, the importers (`/admin/imports`) write a pre-change snapshot under `import_backups/{importId}` so a single bulk import can be **rolled back** via `POST /api/admin/imports/[importId]/restore`.

5.5 Logs

- **Vercel function logs** — primary source of runtime errors. View at `vercel logs <deployment-url>` or in the Vercel dashboard.
- **Firebase Auth logs** — sign-in failures, token verification, in Firebase console.
- **In-app error responses** never leak Firebase / stack traces to the client (per project rules); always log full errors via `console.error` server-side.

5.6 Monitoring

- `@vercel/analytics` and `@vercel/speed-insights` are installed and pull web-vitals automatically.
- The diagnostics page (`/admin/diagnostics`) is the in-app health surface.
- For uptime/external probing, point any external monitor at `GET /api/admin/diagnostics` (gated; returns 401 if cookie missing, which is itself a signal the app is up) or simply `GET /`.

6. Deployment Guide

6.1 Supported targets

- **Vercel** (primary — Fluid Compute, Node.js runtime). Build is `next build`; functions default to Node, no Edge runtime in use.
- Bare-metal Node 20 + `npm run build && npm run start` works for staging mirrors but is not the production target.
- A `Dockerfile` is **not** included.

6.2 Production environment variables

Same as §2.3, with these production-specific notes:

Variable	Type	Default	Impact if missing
<code>FIREBASE_PROJECT_ID</code>	string	none	Boot crash.
<code>FIREBASE_CLIENT_EMAIL</code>	string	none	Boot crash.
<code>FIREBASE_PRIVATE_KEY</code>	multi-line string with <code>\n</code>	none	Boot crash.
<code>NEXT_PUBLIC_FIREBASE_*</code>	string	none	Client-side Firebase bricks.
<code>NEXT_PUBLIC_GOOGLE_MAPS_API_KEY</code>	string	empty	Maps render blank.
<code>GOOGLE_PLACES_API_KEY</code>	string	empty	<code>/api/reviews/[id]</code> returns empty list.
<code>EMAIL_USER</code> / <code>EMAIL_APP_PASSWORD</code>	string	unset	All email sends silently no-op.
<code>NEXT_PUBLIC_SITE_URL</code>	URL	<code>https://car-coating.vercel.app</code>	Cancel links, sitemap, OG point at the wrong host.
<code>GOOGLE_OAUTH_CLIENT_ID</code> / <code>_SECRET</code>	string	unset	<code>scripts/oauth-create-calendars.ts</code> cannot run.
<code>NEXT_PUBLIC_LINE_CHANNEL_ID</code>	string	unset	LINE button hidden.

Manage with the Vercel CLI:

```
vercel env add FIREBASE_PROJECT_ID production
vercel env pull .env.local          # sync to local
vercel env ls production           # list
```

6.3 Deploying

```
# Preview deploy from current branch:
vercel deploy
# Production deploy:
vercel --prod
```

6.4 Database migration procedure

Firestore is schema-less, so most field additions are safe:

1. Add the field with a sensible default in code.
2. Update / extend the Zod schema in `src/lib/validations.ts`.
3. (Optional) Run a one-off backfill script under `scripts/`. Pattern:

```
npx tsx --env-file=.env.local scripts/backfill-override-flags.ts
```

4. If the field needs a composite index, add it to `firestore.indexes.json` and deploy:

```
firebase deploy --only firestore:indexes
```

5. If access changes, edit `firestore.rules` and deploy:

```
firebase deploy --only firestore:rules
```

6.5 Zero-downtime deploy checklist

- ☐ `npm run lint` clean
- ☐ `npm run build` green
- ☐ `npx vitest run` green
- ☐ `npx playwright test` green
- ☐ Firestore rules (if changed) deployed **before** the app, with a strict-er-than-app rule set so an old deploy can still read.
- ☐ Indexes (if new) deployed **before** the app — index builds can take minutes.
- ☐ `vercel --prod` from the green commit.
- ☐ Smoke test: load `/`, load `/<some-slug>`, log in to `/admin`, load `/admin/bookings`.

6.6 Rollback

Vercel keeps every deployment. To roll back:

```
vercel ls          # find the previous good deployment URL
vercel promote <url> # promote to production
```

If a Firestore migration is involved and irreversible, restore from a managed Firestore export (§5.4) before promoting back.

6.7 SSL / domain

Domains are managed via the Vercel dashboard or `vercel domains`. SSL is automatic. Update `NEXT_PUBLIC_SITE_URL` in production env vars whenever you change the canonical hostname — emails, sitemap, robots, and OG metadata all read from this.

7. Maintenance Guide

7.1 Dependency cadence

- **Next.js / React / TypeScript:** track minor & patch updates monthly. Major upgrades (e.g. Next 16 → 17) require reading `node_modules/next/dist/docs/` and a feature-loop run.
- **Firebase Admin & client SDKs:** track minor monthly; major upgrades require auth flow regression test.
- **Tailwind CSS 4:** patch as released.
- **Zod:** patch only without coordination; major upgrades touch every schema in `src/lib/validations.ts`.

Run `npm outdated` and bump in small batches — the Playwright suite is the safety net.

7.2 Firestore maintenance

- Inquiries and reservations grow unbounded — schedule a quarterly archival job (no built-in cleanup yet).
- KPI counters live under `kpi/{storeId}/daily/{YYYY-MM-DD}` and grow by $O(\text{stores} \times \text{days})$. Acceptable for years; consider rollup if it becomes a concern.
- Composite indexes are tracked in `firestore.indexes.json` — prune unused ones after a deploy that removes the corresponding query.

7.3 Storage cleanup

- Image uploads from `/admin/upload` land under `gs://<bucket>/...`. When a store is deleted, its images are **not** auto-pruned. Use `scripts/check-orphan-data.ts` and `scripts/fix-orphan-data.ts` as the manual sweep.

- `import_backups/{importId}` snapshots accumulate per import. They are small (JSON blobs) but should be pruned annually.

7.4 Performance knobs

- `storeSettings` **cache TTL** = 5 minutes (`store-settings.ts`). Increase if bookings/inquiries become hot.
- **Rate-limit window** = 60 s, with per-route `maxRequests` (reservation 5/min, inquiry 10/min, session 10/min). Tune in `src/lib/rate-limit.ts` and at the call sites.
- **Pagination on** `/admin/inquiries` & `/admin/bookings` — currently client-side; revisit if per-store volume crosses ~5000 records.

7.5 Known limitations

- The rate limiter is **in-memory** (`src/lib/rate-limit.ts`). It resets on cold start and is not shared across Vercel function instances. For production-grade limiting, swap for Upstash Redis.
 - `sanitizeHtml` runs server-side; admin-authored CSS is best-effort — do not invite untrusted authors as `super_admin` or `store_admin`.
 - The booking calendar caches month availability for a few minutes per browser. A confirmed booking by another customer may briefly remain on the calendar until refresh.
 - No dark-mode for the admin UI.
-

8. Troubleshooting

Symptom	Likely cause	Resolution
Login redirects to <code>/login?redirect=...</code> immediately after entering credentials	<code>__session</code> cookie not set — see Vercel logs for <code>Session creation failed</code> . Clock skew or wrong <code>FIREBASE_PRIVATE_KEY</code> formatting (<code>\n</code> not preserved).	Verify env vars; the private key must keep <code>\n</code> escapes intact.
Login response: アカウントが登録されていません	Firebase Auth user has no Firestore <code>users/{uid}</code> record.	Create the user via <code>/admin/users</code> (super admin) or <code>POST /api/admin/setup</code> .
<code>/api/reservation</code> returns <code>Booking must be in the future</code>	Date in past relative to server clock (Asia/Tokyo).	Pick a future date; verify Vercel region/clock.
<code>/api/reservation</code> returns <code>Rate limit exceeded</code> (HTTP 429)	More than 5 req/min from the same IP (rate limiter).	Wait 60 s.
Booking confirmation email never arrives	<code>GMAIL_USER</code> / <code>GMAIL_APP_PASSWORD</code> unset, or Gmail blocked the app password.	Check env, regenerate the app password, look for <code>[reservation <id>] email <i> failed in logs</code> .
Calendar event not created on booking confirm	<code>storeSettings/{storeId}.calendarId</code> empty, or the service account is not invited to the calendar.	Use <code>scripts/oauth-create-calendars.ts</code> / <code>scripts/save-calendar-ids.ts</code> ; verify the service account email has write access.
<code>/admin/...</code> returns 401 from APIs	<code>__session</code> cookie expired (14 days) or Firestore Admin SDK rejected the cookie.	Re-login; check env.
Storefront 404 for a store you just created	Slug typo, or <code>is_active: false</code> .	Toggle <code>is_active</code> .
Pricing block shows wrong tiers	<code>featured_tier_ids</code> references stale legacy IDs.	The builder migrates <code>crystal-keeper</code> → <code>crystal</code> , <code>diamond-keeper</code> → <code>diamond</code> , <code>diamond-keeper-double</code> → <code>dia2</code> automatically; re-save the layout to persist.
Admin sidebar missing items	Logged in as <code>store_admin</code> — only items with <code>storeAdminVisible: true</code> are shown by design.	Log in as <code>super_admin</code> for full access.

Symptom	Likely cause	Resolution
Custom HTML block content stripped on storefront	<code>sanitizeHtml()</code> removed disallowed tags / attributes.	Trim to allowed tags; raw <code><script></code> is intentionally rejected.
<code>/sitemap.xml</code> shows wrong host	<code>NEXT_PUBLIC_SITE_URL</code> pointing at the old domain.	Update env var and redeploy.
<code>/api/reviews/[placeId]</code> empty	<code>GOOGLE_PLACES_API_KEY</code> unset or Place ID wrong on the store.	Set the env, set <code>google_place_id</code> on the store.
Banner HTML mode renders blank	Sanitizer rejected everything.	Inspect <code>src/lib/sanitize.ts</code> allow-list; or use structured mode.
Build fails with <code>firebase-admin</code> import error	Mixed ESM / CJS bundling — usually a stale lockfile.	<code>rm -rf node_modules package-lock.json && npm install</code> .

9. API Reference

All routes live under `src/app/api/**`. Auth model:

- **Public** — no auth, IP-rate-limited.
- **Authed** — requires the `__session` cookie (set by `POST /api/auth/session`). Enforced first at `src/proxy.ts`, then again per-handler via `requireAuth(role?)`.
- **Super admin** — additionally requires `role === 'super_admin'`.

Every error response has shape `{ "error": "<message>" }` and uses 4xx/5xx status. No raw Firebase errors are leaked.

9.1 Auth

POST /api/auth/session — create session cookie

Public (rate-limited 10/min/IP).

```
curl -X POST https://example.com/api/auth/session \
  -H 'Content-Type: application/json' \
  -d '{"idToken": "<firebase-id-token>"}'
```

Response: `200 { "success": true }` and a `Set-Cookie: __session=...; HttpOnly; Secure; SameSite=Lax; Path=/; Max-Age=1209600`. Errors: `400 Missing idToken`, `401 Invalid token`, `403 アカウントが登録されていません` (no Firestore user record), `429 Rate limit exceeded`.

DELETE /api/auth/session — logout

Public. Clears the cookie. Returns `200 { "success": true }`.

POST /api/auth/users — create user

Super admin. Body: `{ email, password, displayName, role, managed_stores? }`.

DELETE /api/auth/users/[uid] — delete user

Super admin.

POST /api/admin/setup — create another super admin

Super admin. Body: `{ email, password, displayName? }`. Creates a Firebase user, sets `super_admin` claims, writes to `users/{uid}`.

9.2 Storefront APIs (public)

GET /api/slots?store=<id>&month=YYYY-MM — month availability

Returns `{ "dates": [...] }`.

GET /api/slots?store=<id>&date=YYYY-MM-DD — day slots

Returns `{ "slots": [...] }`.

POST /api/reservation — create a reservation

Public (rate-limited 5/min/IP).

Body:

```
{
  "type": "visit",
  "storeId": "shinjuku-honten",
  "name": "山田太郎",
  "phone": "09012345678",
  "email": "taro@example.com",
  "notes": "",
  "date": "2026-06-01",
  "time": "10:30",
  "autoConfirm": true,
  "vehicleInfo": "Toyota Prius 2023",
  "selectedCoatings": ["crystal"],
  "selectedOptions": []
}
```

Response: `200 { "id": "<reservationId>" }`. Errors include `400` for each required-field check; `429` for rate limit. Side effects: confirmation email + ICS to customer; staff notification email; optional Google Calendar event when `storeSettings.calendarId` is set; KPI increment on the store's daily counter.

POST /api/cancel/[id]?token=<cancelToken> — cancel a reservation

Public (token-authed). Idempotent; rejects `cancelled` / `completed`. Side effects: deletes the linked calendar event; sends cancel emails.

POST /api/inquiry — submit a contact form

Public (rate-limited 10/min/IP).

Body: { `storeId`, `name`, `phone?`, `email`, `message?`, `selectedTier?`, `vehicleInfo?` }. Email regex is enforced. Returns `200` { `"id": "<inquiryId>", "emailWarnings"?: ["customer-confirmation" | "staff-notification"]` }. The optional `emailWarnings` array tells the form whether confirmation or notification emails partially failed so a fallback notice can be shown.

GET /api/inquiries/[token] — public inquiry lookup

Used by token-authenticated follow-up links from staff replies.

GET /api/reviews/[placeId] — Google Places reviews

Server-side proxy using `GOOGLE_PLACES_API_KEY`.

GET /api/blog — list blog posts

Returns published posts.

POST /api/track — record a tracked phone / link click

Used by `TrackedPhoneLink` / `TrackedLink`. Increments the store's daily KPI.

9.3 V3 store APIs

GET /api/v3/stores — list active stores (public)**GET /api/v3/stores?all=true — list incl. inactive (any authed admin)****POST /api/v3/stores — bulk upsert (super admin)**

Accepts JSON array or `Content-Type: text/csv`. Each store validated with `v3StoreWriteSchema`. Caps: `MAX_BATCH_STORES = 500`.

GET `/api/v3/stores/{storeId}` — read one

PUT `/api/v3/stores/{storeId}` — partial update (super admin)

DELETE `/api/v3/stores/{storeId}` — delete (super admin)

GET `/api/v3/stores/export` — CSV export (super admin)

GET|POST|PUT|DELETE `/api/v3/sub-companies` — sub-companies CRUD (super admin)

POST `/api/v3/campaign` — campaign-defaults write (super admin)

9.4 Admin APIs

All under `/api/admin/**`. The proxy rejects unauthenticated calls; each handler additionally calls `requireAuth(role)`.

Route	Methods	Role	Schema
/api/admin/bookings	GET, PATCH	authenticated	bookingPatchSchema
/api/admin/inquiries	GET, PATCH	authenticated	inquiryPatchSchema
/api/admin/tickets	GET, POST	authenticated	ticketActionSchema
/api/admin/tickets/count	GET	authenticated	—
/api/admin/tickets/export	GET	super_admin	—
/api/admin/store-settings	GET, PUT	authenticated (managed)	—
/api/admin/upload	POST	authenticated (managed)	multipart
/api/admin/banner-presets	GET, POST	authenticated	bannerPresetWriteSchema
/api/admin/banner-presets/[id]	GET, PUT, DELETE	scoped	bannerPresetWriteSchema
/api/admin/blog	GET, POST	super_admin	blogPostWriteSchema
/api/admin/blog/[postId]	GET, PUT, DELETE	super_admin	blogPostWriteSchema
/api/admin/blog/import	POST	super_admin	CSV
/api/admin/defaults	GET, PUT	super_admin	—
/api/admin/defaults/policy	GET, PUT	super_admin	—
/api/admin/defaults/overriding-stores	GET	super_admin	—
/api/admin/master/coating-tiers	GET, PUT	super_admin	—
/api/admin/master/appeal-points	GET, PUT	super_admin	—
/api/admin/homepage	GET, PUT	super_admin	—
/api/admin/sub-companies	GET, POST, PUT, DELETE	super_admin	—
/api/admin/seed-content	POST	super_admin	—

Route	Methods	Role	Schema
/api/admin/template	GET, POST	super_admin	—
/api/admin/custom-html-notice	POST	super_admin	—
/api/admin/diagnostics	GET	super_admin	—
/api/admin/kpi	GET	authenticated (managed)	—
/api/admin/kpi/export	GET	super_admin	CSV
/api/admin/overrides	POST	super_admin	—
/api/admin/revalidate	POST	super_admin	—
/api/admin/stores	GET	authenticated	—
/api/admin/stores/[storeId]	GET, PUT, DELETE	scoped	—
/api/admin/stores/import	POST	super_admin	CSV
/api/admin/stores/import/template	GET	super_admin	—
/api/admin/imports/history	GET	super_admin	—
/api/admin/imports/[importId]	GET	super_admin	—
/api/admin/imports/[importId]/download	GET	super_admin	—
/api/admin/imports/[importId]/restore	POST	super_admin	—
/api/admin/setup	POST	super_admin	—

9.5 Example: confirm a booking

```
curl -X PATCH https://example.com/api/admin/bookings \
-H 'Cookie: __session=...' \
-H 'Content-Type: application/json' \
-d '{
  "reservationId": "abc123",
  "status": "confirmed",
  "confirmChoiceIndex": 0,
  "adminMessage": "ご来店をお待ちしております。"
}'
```

Response: 200 { "success": true }. Errors: 400 (Zod), 401 (missing cookie), 403 (not authorized for this store), 404 (no such reservation).

10. Glossary

Term	Meaning
App Router	Next.js routing system based on the <code>src/app/</code> directory; replaces the older <code>pages/</code> router.
Block	A typed unit of content rendered on a storefront (e.g. <code>hero</code> , <code>pricing</code> , <code>gallery</code> , <code>custom_html</code>).
Block layout / page layout	The ordered list of blocks for a single page, stored as JSON in <code>page_layout</code> .
Coating tier	A coating product configured in master data (Crystal KeePer, Diamond KeePer, etc.).
Cancel token	A random per-reservation UUID stored on the doc; used by <code>/api/cancel/[id]?token=...</code> to authenticate customer cancellations without a session.
Custom claim	A piece of metadata attached to a Firebase user identity — <code>role</code> and <code>managed_stores</code> .
Firestore	Google's NoSQL document database; the canonical store for everything except images.
Global default / override	Site-wide default values; per-store overrides recorded in <code>override_flags</code> .
Inquiry	A contact-form submission, status <code>open</code> / <code>replied</code> / <code>closed</code> .
KPI	Daily counters per store: phone clicks, inquiries, bookings.
Page builder	The <code>/admin/builder</code> UI that drag-and-drops blocks.
Preset	A reusable banner snippet (`scope: global
Proxy	Next.js 16's renamed <code>middleware</code> — <code>src/proxy.ts</code> gates <code>/admin</code> , <code>/api/admin</code> , <code>/api/v3</code> writes.
Reservation	A customer booking, status <code>pending</code> / <code>confirmed</code> / <code>completed</code> / <code>cancelled</code> .
Sanitizer	<code>src/lib/sanitize.ts</code> (HTML) / <code>sanitize-css.ts</code> (CSS) — runs on every admin-authored snippet before rendering.
Session cookie	<code>__session</code> , httpOnly, secure, sameSite=lax, 14-day TTL. Created from a Firebase ID token.
Store	A KeePer PRO SHOP branch; one Firestore document per store.
Sub-company	Parent organisation owning multiple stores (used for hierarchy, branding inheritance).
Sub-store	A child store mounted at <code>/<slug>/<subSlug></code> .
Super admin / store admin	The two user roles. Super admins are platform operators; store admins manage only stores in their <code>managed_stores</code> claim.
Ticket	An internal support ticket between store admins and the super-admin team.

11. Changelog

Version	Date	Summary of changes
0.1.0	2026-05-07	Initial published manual snapshot.

<!-- TODO: maintain this table going forward; suggested commit hook: bump on every release tag. -->

12. Support and Contact

<!-- TODO: fill in support email (e.g. support@your-domain.jp). --> <!-- TODO: fill in issue tracker URL (e.g. GitHub Issues link). --> <!-- TODO: fill in Slack channel or alternative contact. -->

For platform-internal support, store admins can also open a ticket inside the app at `/admin/tickets`. The platform team is notified by email via `sendTicketNotificationEmail()`.